Reliability Engineering and System Safety 231 (2023) 109005

Contents lists available at ScienceDirect

Reliability Engineering and System Safety

journal homepage: www.elsevier.com/locate/ress

System design optimization with mixed subsystems failure dependencies

Mohamed Arezki Mellal [a,*], Enrico Zio [b,c], Sameer Al-Dahidi [d], Naoki Masuyama [e], Yusuke Nojima [e]

[a] LMSS, Faculty of Technology, M'Hamed Bougara University, Boumerdes 35000, Algeria
[b] MINES ParisTech, PSL Research University, CRC, Sophia Antipolis, France
[c] Energy Department, Politecnico di Milano, Milano 20156, Italy
[d] Mechanical and Maintenance Engineering Department, School of Applied Technical Sciences, German Jordanian University, Amman 11180, Jordan
[e] Department of Core Informatics, Graduate School of Informatics, Osaka Metropolitan University, Osaka 5998531, Japan

ARTICLE INFO

Keywords:
System availability
System cost
Mixed failure dependencies
Differential evolution (DE)
Manta ray foraging optimization (MRFO)
Shuffled frog leaping algorithm (SFLA)

ABSTRACT

Systems present dependencies among their components failure behavior, which impact their ultimate availability. Previous works addressed the optimal design of systems in relation to its cost and under given availability constraint, considering identical subsystems failure dependencies. The present paper addresses this problem in a realistic scenario by taking into consideration mixed subsystems failure dependencies. The problem is formulated with reference to a complex bridge network system and a series-parallel system. Three nature-inspired optimization techniques are implemented to solve the problem, namely differential evolution (DE), manta ray foraging optimization (MRFO), and shuffled frog leaping algorithm (SFLA) with constraint handling. A numerical evaluation is performed; the results show that DE outperforms MRFO and SFLA.

1. Introduction

1.1. Background

Components failure dependency must be accounted for designing realistic industrial systems, whose subsystems are logically coupled to meet technical design performance criteria, such as Reliability, Availability, Maintainability, Safety, and Cost (RAMS+C) [1–8]. For instance, multiple failures that result directly from a common or shared root cause (i.e., common cause failure), e.g., severe environmental conditions, failure of a component that provides support or enabling functions to other components in the system (i.e., functional dependency), and failure of a component in a system of logically connected components sharing a common load would increase the probability of failure of the remaining components (i.e., cascading failures) are general classification of failure dependency [9].

The interactions between the failures of the components and subsystems, the level of redundancy, the number of repair teams, and other factors influence the system's performance [1,3,10,11]. Besides the technical constraints (such as cost, weight, and size), failure dependency poses challenging issues. It complicates the design optimization problem [12], requiring sophisticated evaluation and optimization techniques for maximizing/minimizing the performance criteria while satisfying the constraints.

1.2. Literature review

Various solution techniques have been proposed and successfully applied for the design optimization of different configurations of systems whose subsystems are subject to failure dependency.

For instance, in [4], the importance of considering failure dependency in the reliability optimization of industrial systems was investigated. Specifically, the authors studied the design of a redundant system under the effect of redundancy dependency (i.e., a particular type of failure dependency [3]). A failure dependence function was introduced to quantify the redundancy dependency, and four classes of redundancy dependence were investigated, i.e., independence, linear, weak and strong.

Mellal et al. [11] investigated the effect of failure dependency on the design optimization of a series-parallel system with six subsystems. Specifically, the objective was to minimize the system's cost under an availability constraint, while considering four classes of failure dependency among the subsystems, i.e., independence, linear, weak, and

* Corresponding author.
E-mail addresses: mellal.mohamed@gmail.com, mellal.mohamed@univ-boumerdes.dz (M.A. Mellal), sameer.aldahidi@gju.edu.jo (S. Al-Dahidi), masuyama@omu.ac.jp (N. Masuyama), nojima@omu.ac.jp (Y. Nojima).

https://doi.org/10.1016/j.ress.2022.109005
Received 28 April 2022; Received in revised form 19 November 2022; Accepted 22 November 2022
Available online 24 November 2022
0951-8320/© 2022 Elsevier Ltd. All rights reserved.

Nomenclatures

RAMS+C	Reliability, Availability, Maintainability, Safety, and Cost
AA-COA	An Adaptive Cuckoo Optimization Algorithm
RAP	Reliability Allocation Problem
RRAP	Redundancy-Reliability Allocation Problem
GA	Genetic Algorithm
NSGA-II	Non-Dominated Sorting Genetic Algorithm II
NSGA-III	Non-Dominated Sorting Genetic Algorithm III
NRGA	Non-dominated Ranking Genetic Algorithm
MOEA/D	Multi-Objective Evolutionary Algorithm based on Decomposition
MaORRAP	Many Objective RRAP
FPA	Flower Pollination Algorithm
PPA	Plant Propagation Algorithm
PSO	Particle Swarm Optimization
COA	Cuckoo Optimization Algorithm
IA	Immune Algorithm
UGF	Universal Generating Function
IEW-VIKOR	Information Entropy Weight-Vlsekriterijumska Optimizacija I Kompromisno Resenje
DE	Differential Evolution
MRFO	Manta Ray Foraging Optimization
SFLA	Shuffled Frog Leaping Algorithm
CCF	Common Cause Failure
CNC	Computer Numerical Control
$C_s(.)$	System cost
n	Vector of redundancy
r	Vector of repair teams
m	Number of subsystems in the system
n_i	Redundancy at subsystem i, $i = 1,\ldots,m$
r_i	Number of repair teams at subsystem i
C_i^c	Cost of a component at subsystem i
C_i^r	Cost of the repair teams at subsystem i
A_s	System availability
$\Theta(\cdot)$	Expression of the system availability depending on the system configuration
D	Type of failure dependency
L	Linear failure dependency
W	Weak failure dependency
S	Strong failure dependency
$A_{i,D}$	Availability of the subsystem i with the type of failure dependency D of the redundant components
λ_i	Failure rate of a component of subsystem i
μ_i	Repair rate of a component of subsystem i
$A_{s,min}$	Minimum allowable system availability
NFE	Number of Function Evaluations
σ	Standard deviation

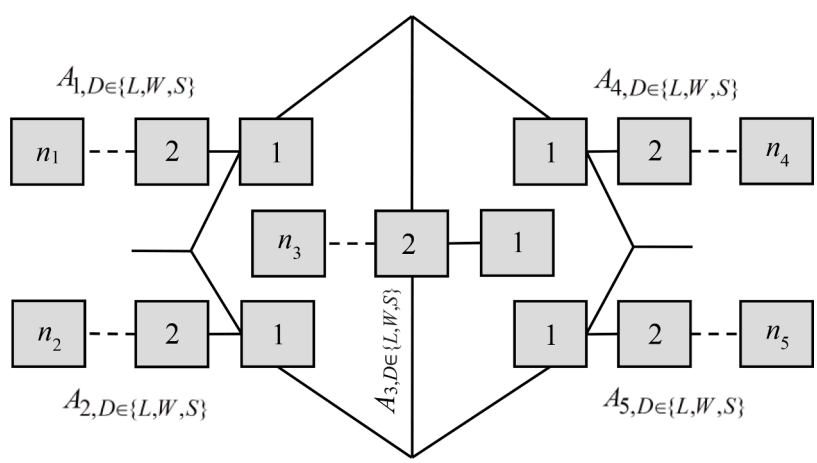

Fig. 1. Complex bridge network system with mixed failure dependencies.

strong. An Adaptive Cuckoo Optimization Algorithm (AA-COA) was proposed for solving the nonlinear optimization problem.

De Paula et al. [13] proposed a new solution approach for the multi-objective Reliability Allocation Problem (RAP) of realistic industrial system configurations whose subsystems were subject to failure dependency. Pareto front, Markov chain, and NSGA-II (Non-Dominated Sorting Genetic Algorithm II) were employed to solve the optimization problem.

Hu et al. [10] investigated the effect of redundancy dependency on the steady-state availability of a repairable series-parallel system with heterogeneous components. The system under investigation contained a variety of components and repairmen. By measuring the failure rates of each subsystem's components, a modified failure dependence function was established to quantify the redundancy dependency. The analysis was carried out using Markov theory and the matrix analysis method to determine the system's steady-state availability.

Li et al. [14] studied the RAP of a spacecraft propulsion phased-mission system with non-exponential components and mixed redundancy strategy (i.e., active and cold standby components). The former was evaluated by resorting to the Semi-Markov process, whereas the types and quantities of the latter were determined by resorting to an improved Genetic Algorithm (GA).

Zhang et al. [15] proposed a minimal cut-based approach to improve the system Reliability-RAP (RRAP), whereas Nath and Muhuri [16] investigated many objective RRAP (MaORRAP) using a novel solution approach based on NSGA type III (NSGA-III) concerning three other popular nature-inspired optimization techniques; however, the availability and failure dependencies have not been addressed.

Mellal et al. [17] investigated the influence of the failure dependency on the multi-objective design optimization of system availability and cost for a series-parallel system. The linear failure dependency among the subsystems was studied. The weighted sum method was employed to convert the multi-objective problem into a single objective one, and the Flower Pollination Algorithm (FPA) and Plant Propagation Algorithm

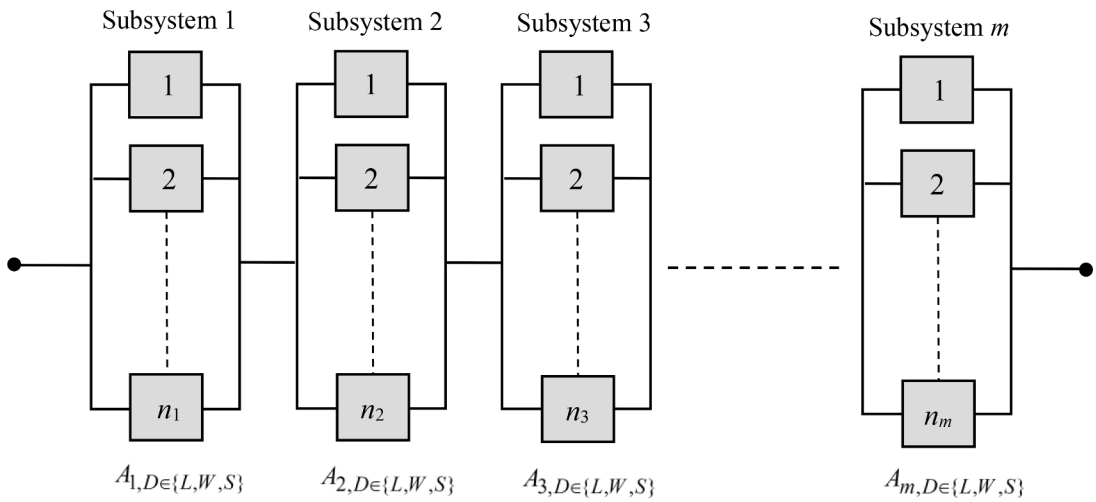

Fig. 2. Series-parallel system with mixed failure dependencies.

Algorithm 1
Pseudo-code of the implemented DE.

```
Initialization;
While Maximum number of
  generations is not reached do
        Evaluate the system cost;
        Round the design variables to integers
    and handling constraints (6) and (8) using
    penalty functions;
        Evolution;
        Mutation;
        Recombination;
        Evolution;
        Selection;
End while
Display the redundancy and repair
  teams vectors, system cost, and
  system availability.
```

Algorithm 2
Pseudo-code of the implemented MRFO.

```
Initialization;
Round the design variables to integers
  and handling constraints (6) and (8)
  using penalty functions;
Update the position of each manta ray;
While Maximum number of generations
  is not reached do
                Generate random values [0,1] to
                perform foraging;
    For Each manta ray do
                Assign a foraging strategy by
                sampling a random value;
                Perform cyclone foraging (spiral
                swarms);
                Perform chain foraging;
                Evaluate the system cost;
                Round the design variables to
                integers and handling constraints (6)
                and (8) using penalty functions;
                Perform somersault foraging;
      Evaluate the system cost;
                Round the design variables to
                integers and handling constraints (6)
                and (8) using penalty functions;
                Update the positions;
    End for
End while
Display the redundancy and repair teams
  vectors, system cost and system
  availability.
```

(PPA) were employed to solve the optimization problem.

Mellal et al. [18] studied the multi-objective availability and cost design optimization problem of a series-parallel system with six subsystems under strong failure dependency. The optimization problem was reduced to a single objective problem by resorting to the weighted sum method, and the resulting optimization problem was solved by using two nature-inspired techniques from the literature, i.e., Particle Swarm Optimization (PSO) and Cuckoo Optimization Algorithm (COA).

Sharifi et al. [19] studied the RRAP of a series-parallel system with multi-state components subjected to Common Cause Failure (CCF) (i.e., an external shock to the system). The optimization problem and the ultimate system's reliability were solved and evaluated by resorting to the Immune Algorithm (IA) meta-heuristic optimization method in conjunction with the Universal Generating Function (UGF).

Zaretalab et al. [6] studied the availability-redundancy allocation problem of a hyper-system composed of various series-parallel subsystems with multi-failure and multi-state components. The multi-objective optimization problem was solved by resorting to four meta-heuristics techniques from the literature, i.e., NSGA-II, NSGA-III, Non-dominated Ranking Genetic Algorithm (NRGA), and Multi-Objective Evolutionary Algorithm based on Decomposition (MOEA/D), combined with the response surface methodology to set up their internal parameters.

Zhang et al. [20] studied the redundancy allocation optimization problem of two parallel-series and two series-parallel systems with interdependent subsystems from statistically dependent components. The optimization problem was approached by a combination of copula-based hierarchical dependent approach (for analyzing the redundancy allocation), Schur-concave copula (for modeling the statistical dependence between the external subsystems), and Archimedean copula (for characterizing the statistical dependence among the inner components).

Gu et al. [21] studied the reliability allocation problem of the Computer Numerical Control (CNC) machine tool with dependent influencing factors (e.g., subsystem complexity, failure frequency, etc.) and subsystem failure dependency. The former was approached by resorting to the gray correlation theory, whereas the latter was handled by the Copula function. The objective weights of the subsystems were determined by resorting to Information Entropy Weight-Vlsekriterijumska Optimizacija I Kompromisno Resenje (IEW--VIKOR) of the literature.

For completeness, Table 1 summarizes the cited works, their brief descriptions, techniques employed/developed, and the type of failure

Algorithm 3
Pseudo-code of the implemented SFLA.

```
Generate a random population;
Evaluate the system cost;
Round the design variables to integers
  and handling constraints (6) and (8)
  using penalty functions;
Find the physical form of the population;
Sort the members (frogs) by
  performance;
While Maximum number of generations
  is not reached do
        Separate the frogs in groups
(memeplexes);
        Evaluate each memeplex and
constraint handling;
        Generate a submemeplex from the
current memeplex;
        Find and update the worst frog
position;
        Sort the memeplexes by
performance;
        Mix the memeplexes and sort the
population;
        Evaluate the system cost;
        Round the design variables to
integers and handle constraints (6) and
(8) using penalty functions;
        Update the best results;
End while
Display the redundancy and repair teams
  vectors, system cost, and system
  availability.
```

dependencies considered in the Authors' works compared to the one at hand in this work.

1.3. Original contribution

From the above-mentioned research works, it is apparent that efforts were mainly dedicated to handling the design optimization problem under identical subsystems failure dependencies, e.g., linear, weak, or strong, using various solution procedures.

In this regard, the original contribution of this work is to address the design optimization problem with mixed subsystems failure dependencies (i.e., linear, weak, or strong). Specifically, the design optimization problem is approached by resorting to three nature-inspired optimization techniques: 1) Differential Evolution (DE), 2) Manta Ray Foraging Optimization (MRFO), and 3) Shuffled Frog Leaping Algorithm (SFLA), whose application results are compared and discussed for two systems' configurations, i.e., a complex bridge network system, and a parallel-series system.

The remainder of the paper is structured as follows. Section 2 formulates the design optimization problem at hand for two systems' configurations. Section 3 presents the three investigated nature-inspired optimization solution approaches. Section 4 illustrates the numerical case studies related to the systems' configurations considered. In Section 5, the results obtained by the application of the three solution approaches are reported and discussed. Finally, some conclusions are drawn in Section 6.

2. Problem formulation

The system availability problem of Refs. [4,11] for the series-parallel systems is addressed as a problem with mixed subsystems failure dependencies, as follows:

$$\text{Minimize} C_s(n,r) = \sum_{i=1}^{m} \left(n_i C_i^c + r_i C_i^r\right) \tag{1}$$

where $C_s(\cdot)$ is the system cost, n is the vector of redundancy, r is the

Table 1
Summary of the cited works and work objective.

Refs.	Brief description	Technique	Type of failure dependencies
Yuan [3]	The redundant dependency and the multiple vacations policy for repairmen of a multi-machine system were studied.	Discrete transform method and the Runge–Kutta method	Identical
Hu et al. [4]	The design of a redundant system under the effect of individual redundancy dependency was investigated.	Dependence function, Markov model, and GA	Identical
Mellal et al. [11]	The effect of various individual types of failure dependency on the design optimization of a series-parallel system with six subsystems was studied.	AA-COA	Identical
De Paula et al. [13]	The multi-objective RAP of realistic industrial system configurations, whose subsystems were subject to failure dependency, was studied.	Pareto front, Markov chain, and NSGA-II	Identical
Hu et al. [10]	The effect of the redundancy dependency on the steady-state availability of a repairable series-parallel system with heterogeneous components was studied.	Markov theory and the Matrix analysis method	Identical
Li et al. [14]	The RAP of a spacecraft propulsion phased-mission system with non-exponential components and a mixed redundancy strategy was studied.	Semi-Markov process and improved GA	Identical
Zhang et al. [15]	The RRAP in a multi-state flow network was studied.	Minimal cut-based approximation scheme	-
Nath and Muhuri [16]	Many objective RRAP (MaORRAP) with various system configurations was investigated.	NSGA-II, NSGA-III, MOEA/D, and SPEA2	-
Mellal et al. [17]	The influence of the linear failure dependency on the multi-objective design optimization of system availability and cost for a series-parallel system was studied.	Weighted sum method, FPA, and PPA	Identical
Mellal et al. [18]	The multi-objective availability and cost design optimization problem of a series-parallel system with six subsystems under strong failure dependency was studied.	Weighted sum method, PSO, and COA	Identical
Sharifi et al. [19]	The RRAP of a series-parallel system with multi-state components	IA and UGF	Identical

(continued on next page)

Table 1 (*continued*)

Refs.	Brief description	Technique	Type of failure dependencies
	subjected to CCF was studied.		
Zaretalab et al. [6]	The availability-redundancy allocation problem of a hyper-system composed of various series-parallel subsystems with multi-failure and multi-state components was studied.	NSGA-II, NSGA-III, NRGA, and MOEA/D combined with Surface Methodology	Identical
Zhang et al. [20]	The RAP of two parallel-series and two series-parallel systems with interdependent subsystems from statistically dependent components was studied.	Copula-based hierarchical dependent approach, Schur-concave copula, and Archimedean copula	Identical
Gu et al. [21]	The RAP of the CNC machine tool with dependent influencing factors and subsystem failure dependency was studied.	Gray correlation theory, Copula function, and IEW-VIKOR	Identical
Present work	**The RRAP with mixed subsystems failure dependencies in a complex bridge network system and a series-parallel system is studied.**	**DE, MRFO, and SELA**	**Mixed**

Table 2
Data of the systems.

Subsystem i	D	λ_i	μ_i	C_i^c	C_i^r	$A_{s,\min}$
1	L	0.07	0.25	30	20	0.90
2	L	0.04	0.12	55	35	0.95
3	W	0.02	0.27	40	25	0.99
4	L	0.03	0.10	75	45	
5	W	0.08	0.15	60	30	
6	S	0.05	0.26	80	60	
7	L	0.01	0.18	70	30	
8	S	0.06	0.30	45	25	
9	L	0.09	0.14	50	30	
10	W	0.05	0.28	25	20	

vector of repair teams, n_i is the redundancy at subsystem i, r_i is the number of repair teams at subsystem i, C_i^c is the cost of a component at subsystem i, C_i^r is the cost of the repair teams at subsystem i, and m is the number of subsystems in the system, subject to,

$$A_s = \Theta\left(A_{i,D\in\{L,W,S\}}\right) \tag{2}$$

where A_s is the system availability, $\Theta(\cdot)$ is the expression of the system availability depending on the system configuration, $A_{i,D\in\{L,W,S\}}$ is the availability of the subsystem i with the type of failure dependency D of the redundant components. L, W, and S denote linear, weak, and strong dependencies, respectively, with,

$$A_{i,D=L} = 1 - \left[1 + \sum_{j=1}^{n_i-r_i} r_i^j \left(\frac{\mu_i}{\lambda_i}\right)^j + \sum_{j=n_i-r_i+1}^{n_i} \frac{r_i^{n_i-r_i} r_i!}{(n_i-j)!}\left(\frac{\mu_i}{\lambda_i}\right)^j\right]^{-1} \tag{3}$$

$$A_{i,D=W} = 1 - \left[1 + \sum_{j=1}^{n_i-r_i} r_i^j (j!)^{-0.5} \left(\frac{\mu_i}{\lambda_i}\right)^j + \sum_{j=n_i-r_i+1}^{n_i} \frac{r_i^{n_i-r_i} r_i!(j!)^{-0.5}}{(n_i-j)!}\left(\frac{\mu_i}{\lambda_i}\right)^j\right]^{-1} \tag{4}$$

Table 3
Results obtained by DE for complex bridge network system.

Run #	C_s	NFE	CPU (s)
	$A_{s,\min}$=0.90		
1	**465**	6000	1177.3172
2	**465**	5000	1375.4833
3	**465**	4500	1294.1577
4	**465**	**3500**	1113.3748
5	**465**	4200	911.3996
6	**465**	5700	1081.5590
7	**465**	5400	1172.1498
8	**465**	4400	1099.8341
9	**465**	4400	1017.9246
10	**465**	3900	1311.6478
	$A_{s,\min}$=0.95		
Run #	C_s	NFE	CPU (s)
1	**520**	3900	1147.4593
2	**520**	**3400**	1306.5662
3	**520**	3800	1516.2809
4	**520**	3500	944.3824
5	**520**	4600	1204.6041
6	**520**	4400	1224.6897
7	**520**	4800	1148.7936
8	**520**	4500	1049.8983
9	**520**	4200	897.0705
10	**520**	5100	895.4735
	$A_{s,\min}$=0.99		
Run #	C_s	NFE	CPU (s)
1	**645**	**3600**	889.0179
2	**645**	5400	885.3774
3	**645**	4800	885.8926
4	**645**	6100	1255.1781
5	**645**	4800	957.7544
6	**645**	4900	1030.8754
7	**645**	4800	1200.5544
8	**645**	4300	1320.4712
9	**645**	6600	1110.6421
10	**645**	6600	1025.6440

$$A_{i,D=S} = 1 - \left[1 + \sum_{j=1}^{n_i-r_i} r_i^j (j!)^{0.5} \left(\frac{\mu_i}{\lambda_i}\right)^j + \sum_{j=n_i-r_i+1}^{n_i} \frac{r_i^{n_i-r_i} r_i!(j!)^{0.5}}{(n_i-j)!}\left(\frac{\mu_i}{\lambda_i}\right)^j\right]^{-1} \tag{5}$$

and,

$$A_{s,\min} \le A_s(n,r) \le 0.9999 \tag{6}$$

$$n_i,\ r_i \in \mathbb{Z}^+ (i = 1, 2, \ldots, m) \tag{7}$$

$$r_i \le n_i\ (i = 1, 2, \ldots, m) \tag{8}$$

where λ_i is the failure rate of a component of subsystem i, μ_i is its repair rate, and $A_{s,\min}$ is the minimum allowable system availability.

2.1. Complex bridge network system

The system availability with mixed failure dependencies of the complex bridge network system shown in Fig. 1 is written as follows:

$$\begin{aligned} A_s &= \Theta\left(A_{i,D\in\{L,W,S\}}\right) \\ &= A_{1,D\in\{L,W,S\}}A_{4,D\in\{L,W,S\}} + A_{2,D\in\{L,W,S\}}A_{5,D\in\{L,W,S\}} \\ &+ A_{2,D\in\{L,W,S\}}A_{3,D\in\{L,W,S\}}A_{4,D\in\{L,W,S\}} + A_{1,D\in\{L,W,S\}}A_{3,D\in\{L,W,S\}}A_{5,D\in\{L,W,S\}} \\ &+ 2A_{1,D\in\{L,W,S\}}A_{2,D\in\{L,W,S\}}A_{3,D\in\{L,W,S\}}A_{4,D\in\{L,W,S\}}A_{5,D\in\{L,W,S\}} \\ &- A_{2,D\in\{L,W,S\}}A_{3,D\in\{L,W,S\}}A_{4,D\in\{L,W,S\}}A_{5,D\in\{L,W,S\}} \\ &- A_{1,D\in\{L,W,S\}}A_{3,D\in\{L,W,S\}}A_{4,D\in\{L,W,S\}}A_{5,D\in\{L,W,S\}} \\ &- A_{1,D\in\{L,W,S\}}A_{2,D\in\{L,W,S\}}A_{4,D\in\{L,W,S\}}A_{5,D\in\{L,W,S\}} \\ &- A_{1,D\in\{L,W,S\}}A_{2,D\in\{L,W,S\}}A_{3,D\in\{L,W,S\}}A_{5,D\in\{L,W,S\}} \\ &- A_{1,D\in\{L,W,S\}}A_{2,D\in\{L,W,S\}}A_{3,D\in\{L,W,S\}}A_{4,D\in\{L,W,S\}} \end{aligned} \tag{9}$$

2.2. Series-parallel system

The system availability with mixed failures dependencies of the

Table 4
Results obtained by MRFO for complex bridge network system.

$A_{s,\min}$=0.90			
Run #	C_s	NFE	CPU (s)
1	**465**	8100	1301.6691
2	**465**	3900	942.6037
3	**465**	2600	933.9337
4	**465**	3000	976.7902
5	485	1200	936.0510
6	475	2400	944.6294
7	**465**	**2100**	994.2598
8	**465**	8600	1370.1289
9	**465**	4000	1743.5406
10	475	8800	1369.1609
$A_{s,\min}$=0.95			
Run #	C_s	NFE	CPU (s)
1	**520**	2600	1536.7764
2	535	2200	1543.7156
3	**520**	2600	1053.8068
4	535	3300	940.7018
5	**520**	4400	939.4955
6	**520**	6500	948.2843
7	**520**	**1200**	939.5308
8	**520**	1700	1066.6953
9	535	2200	1264.1798
10	535	2900	1122.4722
$A_{s,\min}$=0.99			
Run #	C_s	NFE	CPU (s)
1	685	1400	1228.7562
2	660	5300	1139.2593
3	**645**	3800	1015.6790
4	660	9700	962.8741
5	660	4800	1020.6938
6	**645**	**3400**	943.8285
7	700	2900	940.1320
8	660	4100	937.6333
9	675	3600	937.4351
10	685	2100	931.1811

Table 5
Results obtained by SFLA for complex bridge network system.

$A_{s,\min}$=0.90			
Run #	C_s	NFE	CPU (s)
1	720	17700	771.7538
2	715	13200	864.8944
3	655	19500	763.8598
4	650	19100	783.0478
5	765	14100	1106.4180
6	630	12300	788.7146
7	**620**	19200	769.9032
8	635	17900	781.2356
9	640	17500	777.2400
10	705	9000	757.7996
$A_{s,\min}$=0.95			
Run #	C_s	NFE	CPU (s)
1	675	19800	766.5859
2	710	13500	776.0029
3	665	17700	762.5686
4	715	9000	777.5521
5	650	9400	781.0772
6	**645**	13200	774.2439
7	655	16300	774.3847
8	585	16600	784.7384
9	670	20000	769.7558
10	745	18300	776.9470
$A_{s,\min}$=0.99			
Run #	C_s	NFE	CPU (s)
1	740	19800	772.5910
2	780	10900	778.8974
3	**680**	**15100**	778.9331
4	**680**	19800	1010.3950
5	780	14100	782.3487
6	755	19700	763.2788
7	700	19100	1136.0314
8	730	4700	808.4670
9	740	14200	1261.6863
10	750	16600	1203.8918

series-parallel system shown in Fig. 2 with the path and cut method is written as follows:

$$\begin{aligned} A_s &= \Theta\left(A_{i,D\in\{L,W,S\}}\right) \\ &= A_{1,D\in\{L,W,S\}} \times A_{2,D\in\{L,W,S\}} \times A_{3,D\in\{L,W,S\}} \times \ldots \times A_{m,D\in\{L,W,S\}} \\ &= \prod_{i=1}^{m} A_{i,D\in\{L,W,S\}} \end{aligned} \tag{10}$$

3. Solution approaches

To solve the above-described problem, three nature-inspired optimization techniques are implemented with penalty functions to handle the design constraints given in Eqs. (6) and (8). Specifically, the three investigated optimization techniques are: 1) Differential Evolution (DE) (Section 3.1), Manta Ray Foraging Optimization (MRFO) (Section 3.2), and Shuffled Frog Leaping Algorithm (SFLA) (Section 3.3). The motivation for adopting such techniques is due to their effectiveness shown in solving various engineering optimization problems, such as industrial control [22], power flow [23], and reducing harmonic distortion [24], respectively.

3.1. Differential Evolution (DE)

DE is inspired by the same basic concepts of Genetic Algorithm (GA) related to human and animal evolution: genes, chromosomes, population, crossover, mutation, and selection [25] DE is characterized by more perturbations during generation evolution, in an attempt to improve the solution quality. More details on the principles, applications, advantages, and disadvantages of DE are summarized in Refs. [25–31]. The pseudo-code of the implemented DE [27,31] with constraints handling is given in Algorithm 1.

3.2. Manta Ray Foraging Optimization (MRFO)

MRFO algorithm is inspired by the lifestyle of manta rays, notably their foraging strategy [32]. This species of manta has three main foraging strategies: chain, cyclone, and somersault foraging. Mathematical models of these foraging strategies define the steps of this algorithm. Algorithm 2 illustrates the MRFO [32] implemented with constraints handling.

3.3. Shuffled Frog Leaping Algorithm (SFLA)

SFLA is inspired by the leaping and food strategies of frogs [33]. Memeplexes form the groups of the population and the frogs in the group have interactions. An overview of SFLA is given in Refs. [33–36]. The steps of the implemented SFLA [33,35] with constraints handling are illustrated in Algorithm 3.

4. Numerical case studies

Two numerical case studies are considered, i.e., complex bridge network system and series-parallel system with mixed failure dependencies. The data are reported in Table 2, with the series-parallel involving ten subsystems (m=10). Three scenarios of minimum allowable system availability are considered: 0.90, 0.95, and 0.99. The costs, failure rates, and repair rates are given in arbitrary units.

5. Results and discussion

The problem with solution approaches have been implemented using MATLAB 2019a and run on a personal computer I5-7300U vPro 7th Gen (2.7GHz, 8GB of RAM). The population size and the maximum number

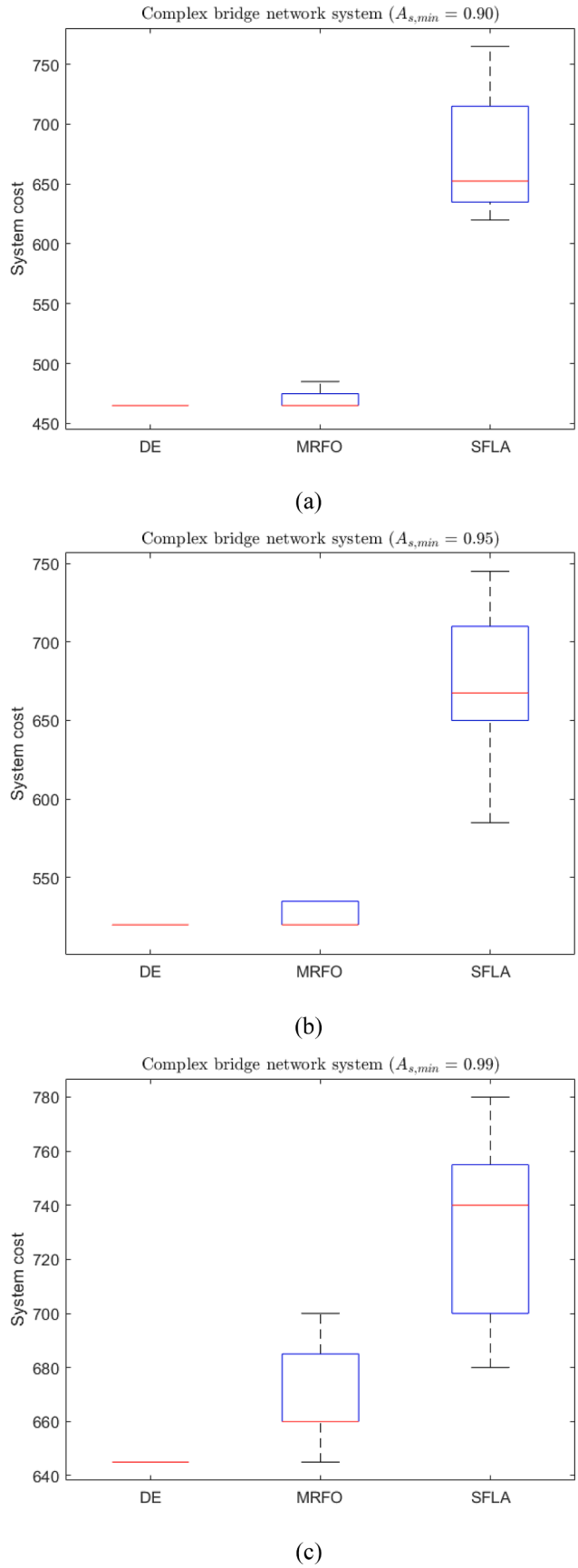

Fig. 3. Box diagram of the optimization techniques in case of complex bridge network system.

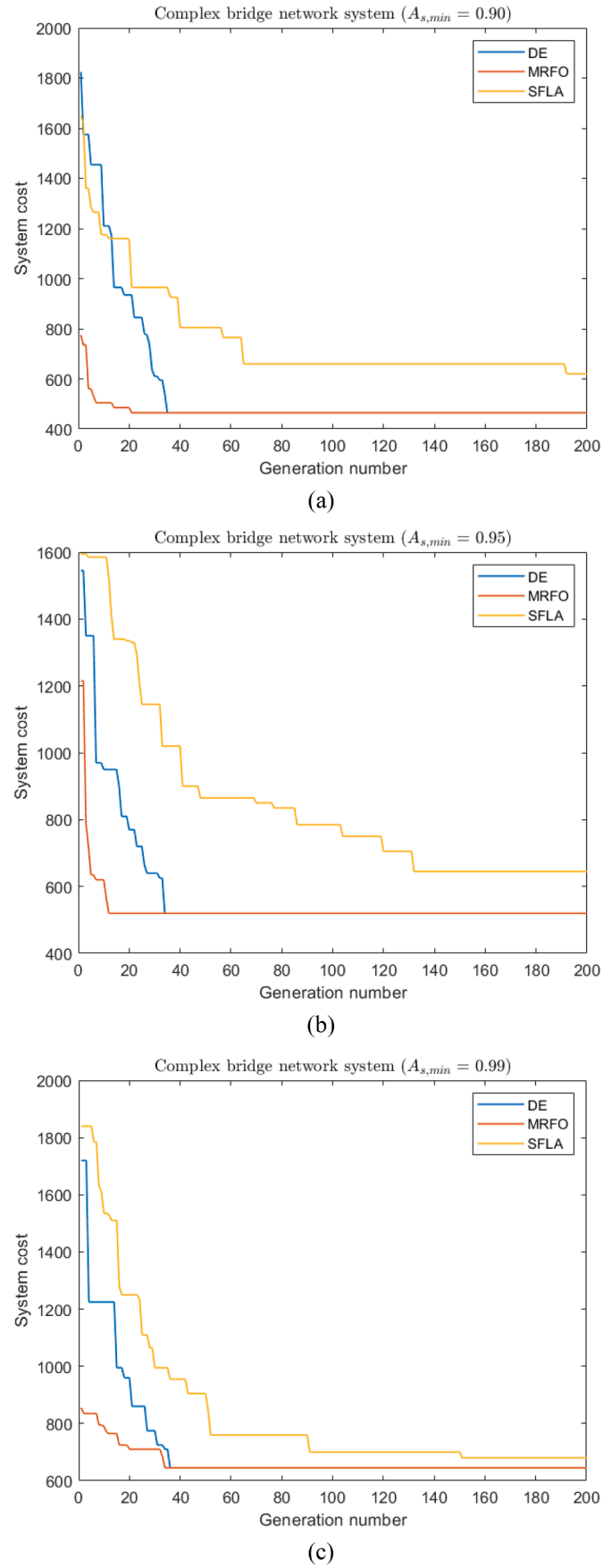

Fig. 4. Convergence of the optimization techniques for the best system cost value in case of complex bridge network system.

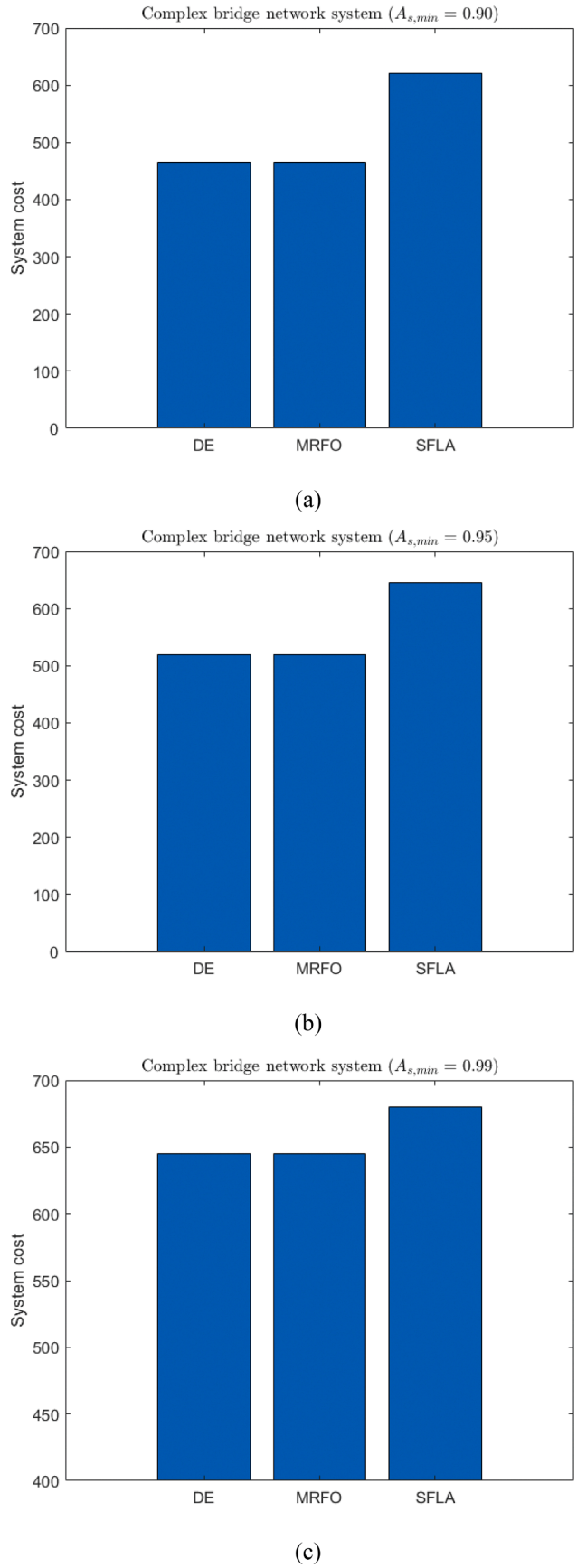

Fig. 5. Best system cost obtained by the optimization techniques for complex bridge network system.

Table 6
Best results obtained by each technique for complex bridge network system.

Technique	n	r	A_s	C_s	NFE	CPU (s)	σ
$A_{s,\min}$=0.90							
DE	(2, 1, 1, 1, 1)	(2, 1, 1, 1, 1)	0.9099	**465**	3500	1113.3748	0
MRFO	(2, 1, 1, 1, 1)	(2, 1, 1, 1, 1)	0.9099	**465**	2100	994.2598	6.6332
SFLA	(3, 2, 1, 1, 2)	(1, 1, 1, 1, 2)	0.9739	620	19200	769.9032	46.3708
$A_{s,\min}$=0.95							
Technique	n	r	A_s	C_s	NFE	CPU (s)	σ
DE	(2, 1, 1, 2, 1)	(1, 1, 1, 1, 1)	0.9617	**520**	3400	1306.5662	0
MRFO	(2, 1, 1, 2, 1)	(1, 1, 1, 1, 1)	0.9617	**520**	1200	939.5308	7.3485
SFLA	(3, 2, 1, 1, 2)	(2, 2, 1, 1, 1)	0.9562	645	13200	774.2439	42.1337
$A_{s,\min}$=0.99							
Technique	n	r	A_s	C_s	NFE	CPU (s)	σ
DE	(3, 1, 1, 3, 1)	(2, 1, 1, 1, 1,)	0.9920	**645**	3600	889.0179	0
MRFO	(3, 1, 1, 3, 1)	(2, 1, 1, 1, 1,)	0.9920	**645**	3400	943.8285	17.2119
SFLA	(3, 1, 1, 2, 2)	(3, 1, 1, 1, 2)	0.9916	680	15100	778.9331	34.6446

of generations are set to 100 and 200, respectively. Each algorithm has been run over ten independent runs for each scenario, and the system cost, required Number of Function Evaluations (NFE), and consumed CPU (s) time until the last generation are reported.

Tables 3–5 report the system cost, required NFE, and consumed CPU time of each optimization technique at each run, for the complex bridge network system in each scenario of minimum allowable system availability, i.e., $A_{s,min}$= 0.90, 0.95, and 0.99. The bold font shown in the Tables highlights the best system cost value, and the minimum NFE in case of similar system cost values.

From the box diagram shown in Fig. 3, one can observe that the statistical results of the system cost obtained by the DE algorithm outperformed those obtained by the MRFO and SFLA, i.e., the former

Table 7
Results obtained by DE for series-parallel system.

Run #	C_s	NFE	CPU (s)
$A_{s,\min}$=0.90			
1	**1945**	15200	2732.4570
2	**1945**	19200	2582.1345
3	**1945**	19500	2686.5100
4	1955	18600	3097.2642
5	**1945**	**13600**	2615.0992
6	**1945**	18200	3118.2216
7	1950	18800	2868.1816
8	1950	18500	2984.3187
9	1950	19000	3118.7751
10	**1945**	16100	2561.1518
$A_{s,\min}$=0.95			
Run #	C_s	NFE	CPU (s)
1	2135	17700	2567.0794
2	2145	19600	3416.0617
3	2145	19400	3443.8631
4	2145	19800	2828.4706
5	2150	16800	2548.8626
6	2145	14900	2594.8390
7	2150	16800	3321.2868
8	2135	16300	2846.8430
9	**2130**	18300	3095.6115
10	2150	18500	2784.6178
$A_{s,\min}$=0.99			
Run #	C_s	NFE	CPU (s)
1	2630	18700	3017.4938
2	**2610**	15300	2644.9113
3	2635	17700	2652.1003
4	2635	18300	3024.3520
5	2630	15800	3049.6619
6	2625	19000	2984.4698
7	2625	20000	2526.0460
8	2625	19400	3165.4934
9	2635	18700	3048.1317
10	2625	16700	3666.8479

Table 8
Results obtained by MRFO for series-parallel system.

Run #	C_s	NFE	CPU (s)
	$A_{s,\min}$=0.90		
1	**1955**	6400	5410.7312
2	1985	8900	2808.0915
3	2050	3800	2500.6277
4	1990	5900	2544.6862
5	1960	7700	2513.9964
6	1960	5300	2449.4456
7	1990	6100	3338.3884
8	1975	9900	2925.9040
9	2020	9900	3497.6256
10	1985	5200	2674.1586
	$A_{s,\min}$=0.95		
Run #	C_s	NFE	CPU (s)
1	2275	8800	2541.9918
2	2175	9000	2601.4976
3	2245	8000	2598.2250
4	2285	4500	2701.0869
5	2185	7800	3043.3096
6	2170	7000	3152.5794
7	**2165**	8800	2933.5055
8	2295	9000	3960.3859
9	2175	9000	3689.8909
10	2235	9900	3078.9573
	$A_{s,\min}$=0.99		
Run #	C_s	NFE	CPU (s)
1	**2615**	9300	2991.0308
2	**2615**	**6900**	2973.3274
3	2625	5100	2880.5267
4	2640	8600	2929.5727
5	2625	9000	2755.2140
6	2650	9200	2684.3754
7	2685	6000	3492.2985
8	2720	9400	3369.4562
9	2715	5700	3732.2819
10	2625	5500	2645.0717

Table 9
Results obtained by SFLA for series-parallel system.

Run #	C_s	NFE	CPU (s)
	$A_{s,\min}$=0.90		
1	2570	19200	2640.4102
2	2865	19800	2575.5690
3	2780	15700	2990.8184
4	**2325**	19600	2996.0632
5	2560	19900	2710.7286
6	2760	17100	2842.8695
7	2770	20000	2748.2211
8	2630	14300	2402.1427
9	2435	19500	3119.3890
10	2460	19200	2451.5397
	$A_{s,\min}$=0.95		
Run #	C_s	NFE	CPU (s)
1	2750	14700	3060.8204
2	**2600**	15000	2908.4473
3	2750	18600	3091.1952
4	2825	16400	6049.4774
5	2775	15000	4784.1460
6	2775	16200	2642.4832
7	2895	19500	2542.9502
8	2890	18700	2401.7851
9	3150	19900	2571.5257
10	2845	18200	3035.8387
	$A_{s,\min}$=0.99		
Run #	C_s	NFE	CPU (s)
1	2895	15600	2950.6115
2	2945	17500	3171.5176
3	3240	18600	2536.1910
4	3005	17400	3126.7395
5	3035	16400	2779.0879
6	3145	16300	3385.6827
7	**2865**	18200	3098.6899
8	3100	16800	2925.1477
9	2915	15800	3533.6858
10	2995	19500	2754.1686

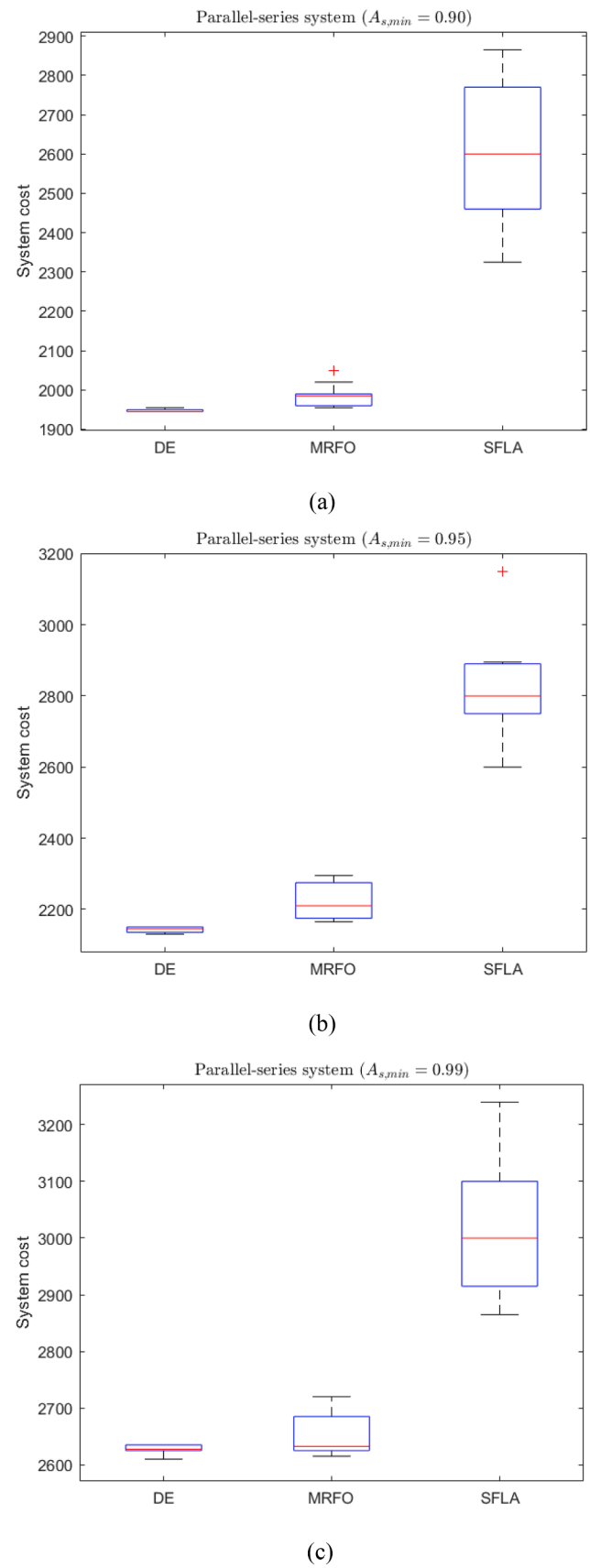

Fig. 6. Box diagram of the optimization techniques in case of series-parallel system.

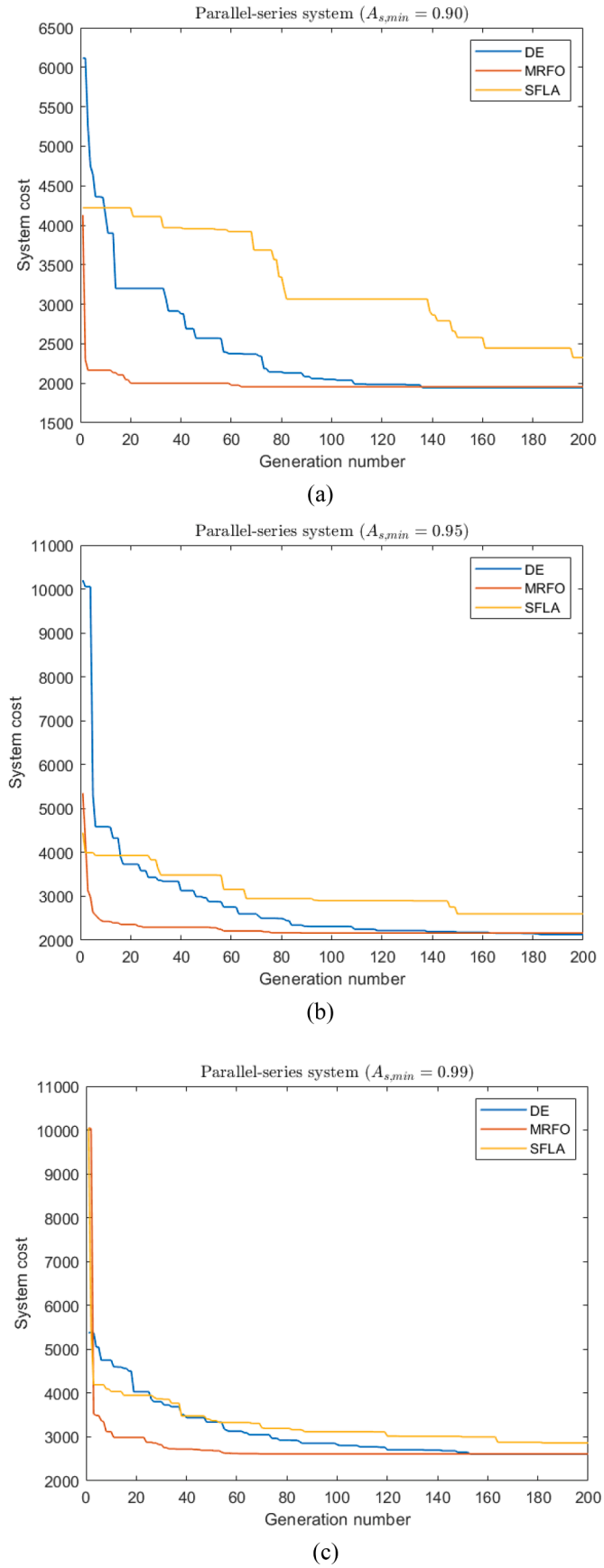

Fig. 7. Convergence of the optimization techniques for the best system cost value in case of series-parallel system.

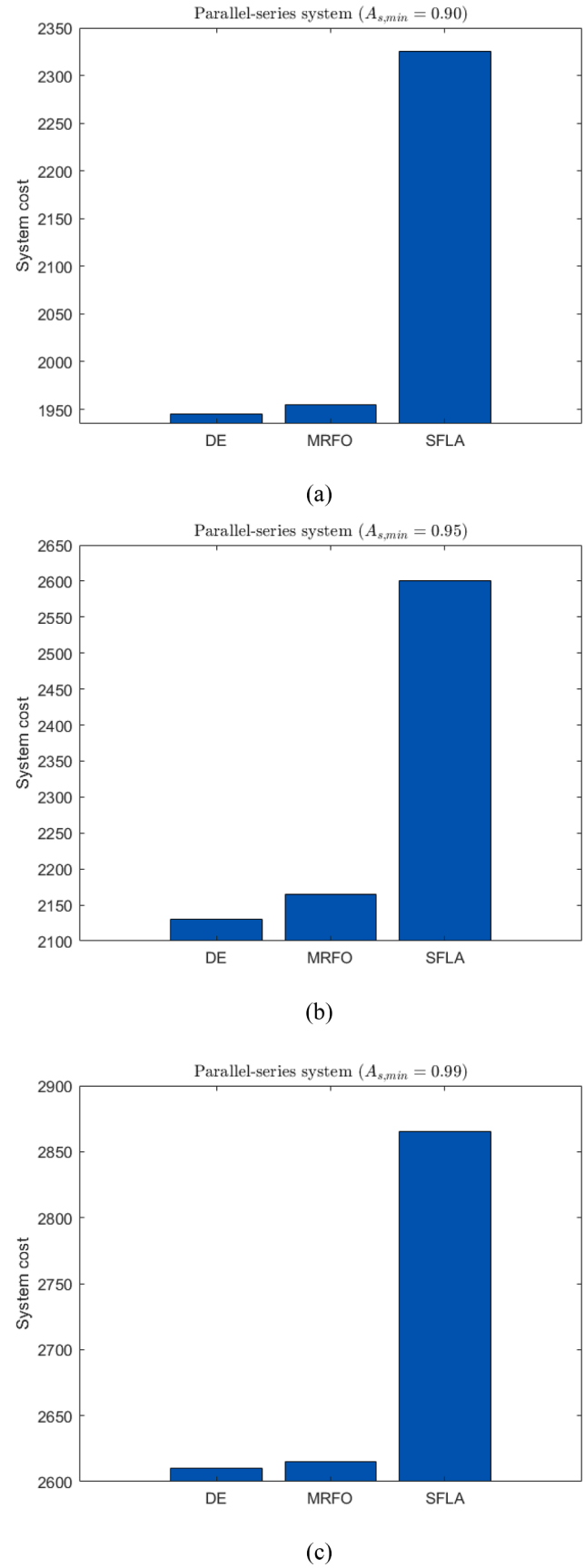

Fig. 8. Best system cost obtained by the optimization techniques for series-parallel system.

Table 10
Best results obtained by each technique for series-parallel system.

Technique	n	r	A_s	C_s	NFE	CPU (s)	σ
			$A_{s,\min}$=0.90				
DE	(3, 3, 2, 3, 4, 2, 2, 3, 4, 3)	(2, 2, 1, 1, 2, 1, 1, 1, 2, 1)	0.9007	**1945**	13600	2615.0992	3.3541
MRFO	(3, 3, 2, 3, 3, 2, 2, 2, 4, 3)	(2, 2, 2, 2, 2, 1, 1, 2, 2, 2)	0.9007	1955	6400	5410.7312	27.7669
SFLA	(6, 3, 2, 2, 5, 3, 2, 2, 5, 3)	(4, 2, 2, 2, 2, 1, 2, 1, 4, 2)	0.9185	2325	19600	2996.0632	167.5030
			$A_{s,\min}$=0.95				
Technique	n	r	A_s	C_s	NFE	CPU (s)	σ
DE	(3, 3, 2, 3, 4, 3, 2, 2, 4, 3)	(2, 2, 1, 2, 3, 1, 1, 2, 3, 2)	0.9514	**2130**	18300	3095.6115	6.7823
MRFO	(3, 3, 2, 3, 4, 3, 2, 3, 4, 3)	(2, 2, 1, 3, 3, 1, 1, 1, 2, 2)	0.95	2165	8800	2933.5055	49.5202
SFLA	(6, 5, 2, 3, 4, 2, 2, 6, 4, 4)	(2, 2, 2, 3, 2, 2, 2, 3, 2, 3)	0.9520	2600	15000	2908.4473	134.9157
			$A_{s,\min}$=0.99				
Technique	n	r	A_s	C_s	NFE	CPU (s)	σ
DE	(4, 4, 3, 4, 5, 3, 3, 3, 5,4)	(2, 2, 1, 2, 4, 1, 1, 2, 3, 2)	0.9905	**2610**	15300	2644.9113	7.1589
MRFO	4, 4, 3, 4, 5, 3, 2, 3, 5, 4)	(2, 2, 2, 2, 4, 1, 2, 2, 3, 3)	0.9901	2615	6900	2973.3274	38.4090
SFLA	(4, 4, 3, 4, 5, 3, 2, 4, 5, 4)	(3, 3, 2, 3, 3, 1, 2, 1, 3, 2)	0.9900	2865	18200	3098.6899	112.8893

provided lowest and stable system costs compared to those latter.

Fig. 4 shows the convergence profile of the three optimization techniques for the best system cost values in the case of complex bridge network system, whereas Fig. 5 compares the best system cost values of each optimization technique, at the three different scenarios of the minimum allowable system availability. Looking at Figs. 4 and 5, one can recognize the fast convergence capability of the DE algorithm compared to the MRFO and SFLA and the small system's cost statistical values acquired, respectively.

Table 6 summarizes the corresponding optimal design variables, system availability, system cost, required NFE, CPU, and standard deviation (σ) for each optimization technique. It can be observed that the system costs obtained by DE and MRFO are similar in all scenarios: 465, 520, and 645, in an arbitrary unit, respectively. The system costs obtained by SFLA are 620, 645, and 680, in an arbitrary unit, respectively. The systems costs obtained by DE and MRFO are lower than those of SFLA. DE has required more NFE and consumed more CPU time than MRFO, in all scenarios except in case A_s,$_{\min}$=0.99. However, the σ of DE is 0 in all scenarios (i.e., stable performance), compared to MRFO, and equal to 6.6332, 7.3485, and 17.2119, respectively.

Tables 7–9 report the system cost, required NFE, and consumed CPU time of each optimization technique at each run, for the series-parallel system in each scenario.

Similarly, Fig. 6 shows that the box diagram of DE algorithm is better than those of MRFO and SFLA in terms of system cost's statstical values acquired and its stable performance, at the three different scenarios of the minimum allowable system availability.

Fig. 7 shows the convergence profile of the three optimization techniques for the best system cost value in the case of series-parallel system, whereas Fig. 8 compares the best system cost values of each optimization technique, at the three different scenarios of the minimum allowable system availability. Similarly, one can recognize the fast convergence capability of the DE algorithm compared to the MRFO and SFLA and the small system's cost statistical values acquired, respectively.

The corresponding optimal design variables, system availability, system cost, required NFE, CPU, and σ of each optimization technique are reported in Table 10. From Table 10, it can be observed that DE has outperformed MRFO and SFLA in all scenarios of minimum system availability (0.90, 0.95, and 0.99). The system costs provided by DE are 1945, 2130, and 2610, in an arbitrary unit, whereas those of MRFO are 1955, 2165, and 2615 and those of SFLA are 2325, 2600, and 2865. This means that the best system costs were provided by DE, as illustrated in Figs. 6, 7 and 8. Table 10 also shows that the best statistical results of the system's costs are given by DE. Although MRFO required the fewest NFE, DE has the smallest SD. The overall results from both case studies lead to conclude that DE has outperformed MRFO and SFLA in solving the described problem.

6. Conclusions and future works

The goal of this work was to address the problem of system cost optimization under system availability constraint, considering mixed subsystems failure dependencies of three types: linear, weak, and strong. The system configurations considered are complex bridge network and series-parallel systems. However, the problem formulated can be applied to any system configuration. Differential Evolution (DE), Manta Ray Foraging Optimization (MRFO), and Shuffled Frog Leaping Algorithm (SFLA) with constraint handling have been implemented to solve the problem, and their application results are compared and discussed. The numerical case studies reveal the superiority of DE due to the diversity option during the exploration of the generations. Future works will address the multi-objective formulation of the problem being studied in this work.

CRediT authorship contribution statement

Mohamed Arezki Mellal: Conceptualization, Methodology, Software, Validation, Writing – original draft. **Enrico Zio:** Methodology, Validation, Writing – original draft. **Sameer Al-Dahidi:** Methodology, Validation, Writing – original draft. **Naoki Masuyama:** Methodology, Validation, Writing – original draft. **Yusuke Nojima:** Methodology, Validation, Writing – original draft.

Declaration of Competing Interest

We wish to confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

Data availability

No data was used for the research described in the article.

References

[1] Yu H, Chu C, Châtelet E, Yalaoui F. Reliability optimization of a redundant system with failure dependencies. Reliab Eng Syst Saf 2007;92:1627–34. https://doi.org/10.1016/j.ress.2006.09.015.
[2] Siju KC, Kumar M. System reliability estimation and cost analysis of series-parallel systems in the presence of repair dependence function. Int J Reliab Saf 2016;10:48–71.
[3] Yuan L. Reliability analysis for a k-out-of-n:G system with redundant dependency and repairmen having multiple vacations. Appl Math Comput 2012;218:11959–69. https://doi.org/10.1016/j.amc.2012.06.006.
[4] Hu L, Yue D, Li J. Availability analysis and design optimization for a repairable series-parallel system with failure dependencies. Int J Innov Comput Inf Control 2012;8:6693–705. https://doi.org/10.4156/aiss.vol3.

[5] Yu H, Chu C, Châtelet É. Availability optimization of a redundant system through dependency modeling. Appl Math Model 2014;38:4574–85. https://doi.org/10.1016/j.apm.2014.03.006.
[6] Zaretalab A, Sharifi M, Guilani PP, Taghipour S, Niaki STA. A multi-objective model for optimizing the redundancy allocation, component supplier selection, and reliable activities for multi-state systems. Reliab Eng Syst Saf 2022;222:108394. https://doi.org/10.1016/J.RESS.2022.108394.
[7] Li S, Chi X, Yu B. An improved particle swarm optimization algorithm for the reliability–redundancy allocation problem with global reliability. Reliab Eng Syst Saf 2022;225:108604. https://doi.org/10.1016/J.RESS.2022.108604.
[8] Liu M, Wang D, Zhao J, Si S. Importance measure construction and solving algorithm oriented to the cost-constrained reliability optimization model. Reliab Eng Syst Saf 2022;222:108406. https://doi.org/10.1016/J.RESS.2022.108406.
[9] Humphreys P, Jenkins AM. Dependent failures developments. Reliab Eng Syst Saf 1991;34:417–27. https://doi.org/10.1016/0951-8320(91)90111-J.
[10] Hu L, Yue D, Ma Z. Availability analysis of a repairable series-parallel system with redundant dependency. J Syst Sci Complex 2020;33:446–60. https://doi.org/10.1007/s11424-019-8039-x.
[11] Mellal MA, Zio E. An adaptive cuckoo optimization algorithm for system design optimization under failure dependencies. Proc Inst Mech Eng Part O J Risk Reliab 2019. https://doi.org/10.1177/1748006x19863639. 1748006x1986363.
[12] Lin YH, Li YF, Zio E. Reliability assessment of systems subject to dependent degradation processes and random shocks. Https://DoiOrg/101080/0740817X20161190481 2016;48:1072–85. doi:10.1080/0740817X.2016.1190481.
[13] de Paula CP, Visnadi LB, de Castro HF. Multi-objective optimization in redundant system considering load sharing. Reliab Eng Syst Saf 2019;181:17–27. https://doi.org/10.1016/j.ress.2018.08.012.
[14] Li XY, Li YF, Huang HZ. Redundancy allocation problem of phased-mission system with non-exponential components and mixed redundancy strategy. Reliab Eng Syst Saf 2020;199:106903. https://doi.org/10.1016/J.RESS.2020.106903.
[15] Zhang H, Sun M, Li YF. Reliability–redundancy allocation problem in multi-state flow network: minimal cut-based approximation scheme. Reliab Eng Syst Saf 2022;225:108564. https://doi.org/10.1016/J.RESS.2022.108564.
[16] Nath R, Muhuri PK. Evolutionary optimization based solution approaches for many objective reliability-redundancy allocation problem. Reliab Eng Syst Saf 2022;220:108190. https://doi.org/10.1016/J.RESS.2021.108190.
[17] Mellal MA, Salhi A, Márquez FPG. System availability and cost optimization under failure dependencies by flower pollination and plant propagation algorithms. In: Fifteenth Int. Conf. Manag. Sci. Eng.78. Springer; 2021. p. 469–76. https://doi.org/10.1007/978-3-030-79203-9_36.
[18] Mellal MA, Zio E. Multi-objective availability and cost optimization by PSO and COA for series-parallel systems with subsystems failure dependencies. Microprocess Microsyst 2022;89:104422. https://doi.org/10.1016/j.micpro.2021.104422.
[19] Sharifi M, Sayyad A, Taghipour S, Abhari A. Optimizing a joint reliability-redundancy allocation problem with common cause multi-state failures using immune algorithm. J Risk Reliab 2022. https://doi.org/10.1177/1748006x221078128.
[20] Zhang J, Yan R, Wang J. Reliability optimization of parallel-series and series-parallel systems with statistically dependent components. Appl Math Model 2022;102:618–39. https://doi.org/10.1016/J.APM.2021.10.003.
[21] Gu D, Xu Z, Zhong Y, Li Q, Long Z. Reliability allocation method of comprehensive weight computer numerical control machine tool based on failure correlation and factor correlation. Qual Reliab Eng Int 2022. https://doi.org/10.1002/QRE.3124.
[22] Nadimi-Shahraki MH, Taghian S, Mirjalili S, Faris H. MTDE: An effective multi-trial vector-based differential evolution algorithm and its applications for engineering design problems. Appl Soft Comput 2020;97:106761. https://doi.org/10.1016/J.ASOC.2020.106761.
[23] Guvenc U, Bakir H, Duman S, Ozkaya B. Optimal power flow using manta ray foraging optimization. Lect. Notes Data Eng. Commun. Technol., 76. Springer Science and Business Media Deutschland GmbH; 2021. p. 136–49. https://doi.org/10.1007/978-3-030-79357-9_14/FIG.S/7.
[24] Darvishi A, Alimardani A, Vahidi B, Hosseinian SH. Shuffled frog-leaping algorithm for control of selective and total harmonic distortion. J Appl Res Technol 2014;12:111–21. https://doi.org/10.1016/S1665-6423(14)71611-6.
[25] Storn R, Price K. Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces. J Glob Optim 1997;11:341–59. https://doi.org/10.1023/A:1008202821328.
[26] Storn R, Price K. Differential evolution – A simple and efficient adaptive scheme for global optimization over continuous spaces. 1995. Berkeley, CA, USA.
[27] Karaboga N, Cetinkaya B. Performance comparison of genetic and differential evolution algorithms for digital FIR filter design. Adv. Inf. Syst. 2004:482–8.
[28] Price K V. Differential evolution. Handb. Optim., 38. Berlin, Heidelberg: Springer; 2013. p. 187–214. https://doi.org/10.1007/978-3-642-30504-7_8.
[29] Das S, Suganthan PN. Differential evolution: A survey of the state-of-the-art. IEEE Trans Evol Comput 2011;15:4–31. https://doi.org/10.1109/TEVC.2010.2059031.
[30] Mellal MA, Zio E. Availability optimization of parallel-series system by evolutionary computation. In: 3rd Int. Conf. Syst. Reliab. Saf.; 2018. p. 198–202. https://doi.org/10.1109/ICSRS.2018.8688722.
[31] Mellal MA, Salhi A. Parallel–Series System Optimization by Weighting Sum Methods and Nature-Inspired Computing, 2020, p. 231–51. doi:10.1007/978-981-13-9263-4_10.
[32] Zhao W, Zhang Z, Wang L. Manta ray foraging optimization: An effective bio-inspired optimizer for engineering applications. Eng Appl Artif Intell 2020;87:103300. https://doi.org/10.1016/J.ENGAPPAI.2019.103300.
[33] Eusuff M, Lansey K, Pasha F. Shuffled frog-leaping algorithm: A memetic meta-heuristic for discrete optimization. Eng Optim 2006;38:129–54.
[34] Rajeev Gandhi BG, Bhattacharjya RK. Introduction to shuffled frog leaping algorithm and its sensitivity to the parameters of the algorithm. Nature-inspired methods metaheuristics optim. Springer; 2020. p. 105–17.
[35] Mellal MA, Frik A, Boutiche R. Reliability optimization of power plant safety system using grey wolf optimizer and shuffled frog-leaping algorithm. In: Mellal MA, Pecht MG, editors. Nature-Inspired Comput. Paradig. Syst. Reliab. Availability, Maint. Saf. Cost Progn. Heal. Manag. Academic Press; 2021. p. 1–13. https://doi.org/10.1016/B978-0-12-823749-6.00008-8.
[36] Liu Y, Heidari AA, Ye X, Chi C, Zhao X, Ma C, et al. Evolutionary shuffled frog leaping with memory pool for parameter optimization. Energy Reports 2021;7:584–606. https://doi.org/10.1016/J.EGYR.2021.01.001.